

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3067 Redes

Kevin Velásquez



Proyecto #1 - Entrega 1

Ihan Marroquin - 23108

GUATEMALA, 20 de agosto del 2026

Inciso 8. Especificación del servidor MCP desarrollado.

El servidor industrial-maintenance-server implementa un caso de gestión de mantenimiento para una planta de bebidas. El chatbot puede consultar maquinaria, repuestos y órdenes de trabajo; las acciones que modifican datos requieren confirmación del usuario desde la aplicación anfitriona.

1. Especificación general del servidor MCP local.

a. Nombre, versión y protocolo.

- i. El servidor se identifica como industrial-maintenance-server, versión 1.0.0. Implementa manualmente la revisión MCP 2025-11-25 sobre mensajes JSON-RPC 2.0, sin utilizar FastMCP ni un SDK que oculte el intercambio.

b. Transporte y codificación.

- i. Se ejecuta como un subproceso local y utiliza stdin y stdout. Cada mensaje es un objeto JSON codificado en UTF-8, delimitado por una nueva línea. stdout se reserva para mensajes válidos del protocolo y stderr para diagnósticos.

c. Persistencia.

- i. Los registros se almacenan en data/maintenance.json. El conjunto inicial contiene tres máquinas, tres repuestos y una orden de trabajo. Las escrituras se realizan primero en un archivo temporal y después se reemplaza el archivo principal.

d. Caso de uso industrial.

- i. El servidor centraliza consultas de estado, inventario de repuestos y administración de órdenes de mantenimiento para una pequeña planta de bebidas. El LLM selecciona herramientas, pero la lógica y la modificación de datos permanecen en el servidor.

2. Puntos de interacción y ciclo de vida.

a. initialize.

- i. Recibe protocolVersion, capabilities y clientInfo. Devuelve la versión negociada, la capacidad tools, la información del servidor y una instrucción para consultar antes de modificar.

b. notifications/initialized.

- i. Es la notificación del cliente que completa la inicialización. Antes de recibirla, el servidor rechaza las operaciones normales con el error -32002.

c. ping.

- i. Comprueba disponibilidad y responde con un objeto de resultado vacío.

d. tools/list.

- i. Devuelve las seis herramientas, sus descripciones, esquemas de entrada y anotaciones de lectura o escritura.

e. tools/call.

- i. Recibe name y arguments, valida los parámetros e invoca la operación. El resultado incluye content, structuredContent e isError.

- f. Endpoints.**
 - i. Por ser un servidor local sobre stdio no expone URL ni endpoint HTTP. Sus puntos de interacción son los métodos MCP descritos anteriormente.
- 3. Herramientas y parámetros.
 - a. list_machines.**
 - i. Lista la maquinaria registrada. status es opcional y acepta operational, warning, stopped o maintenance. No modifica información.
 - b. get_machine_status.**
 - i. Devuelve el registro completo de una máquina. machine_id es obligatorio; un ejemplo válido es SELL-01. No modifica información.
 - c. list_work_orders.**
 - i. Consulta órdenes de mantenimiento. machine_id es opcional y filtra por máquina; status también es opcional y acepta open o closed.
 - d. check_spare_part.**
 - i. Busca por número de parte, nombre o máquina compatible. query es obligatorio y no puede estar vacío. El resultado indica existencia, ubicación y si se recomienda reabastecer.
 - e. create_work_order.**
 - i. Crea y persiste una orden. Requiere machine_id, issue de al menos cinco caracteres y priority con uno de los valores low, medium, high o critical. La aplicación anfitriona solicita confirmación antes de invocarla.
 - f. close_work_order.**
 - i. Cierra una orden abierta y registra su solución. Requiere work_order_id y resolution de al menos cinco caracteres. También solicita confirmación por ser una operación de escritura.
- 4. Validación, resultados y registro de interacciones.
 - a. Validación de entrada.**
 - i. El servidor rechaza parámetros adicionales, valores fuera de los catálogos, identificadores desconocidos y cadenas menores a la longitud requerida. Los errores de herramienta se devuelven con isError igual a true.
 - b. Control del usuario.**
 - i. El anfitrión inspecciona las anotaciones y el nombre de la herramienta. Antes de crear o cerrar información muestra servidor, herramienta y argumentos, y permite aprobar o cancelar.
 - c. Registro JSON Lines.**
 - i. Las solicitudes, notificaciones, respuestas y diagnósticos se guardan con marca UTC, nombre del servidor y dirección en logs/mcp_interactions.jsonl.

5. Instalación, ejecución y ejemplo de uso.

a. Ejecución independiente.

- i. Desde la raíz de entrega_1_mcp se inicia con `python -m src.maintenance_server`. El servidor propio utiliza únicamente la biblioteca estándar de Python.

b. Pruebas automatizadas.

- i. La orden `python -m unittest discover -s tests -v` valida servicio, persistencia, inicialización, descubrimiento, llamadas y registro. Las seis pruebas incluidas finalizaron correctamente durante esta revisión.

c. Ejemplo JSON-RPC.

- i. Para consultar SELL-01 se envía una solicitud `tools/call` con `name` igual a `get_machine_status` y `arguments` igual a `{machine_id: SELL-01}`; la respuesta devuelve el registro de la máquina dentro del resultado de la herramienta.

Inciso 10. Conclusiones y comentario sobre el proyecto.

1. Conclusiones.

a. Separación entre razonamiento y ejecución.

- i. MCP funciona como una capa intermedia entre el modelo de lenguaje y las herramientas externas. El modelo interpreta la solicitud y selecciona una herramienta, mientras que el servidor ejecuta la acción con parámetros definidos y devuelve un resultado verificable.

b. Utilidad de JSON-RPC 2.0.

- i. Los identificadores permiten relacionar cada solicitud con su respuesta. Además, separar `stdout` para el protocolo y `stderr` para diagnósticos evita interrumpir el transporte por `stdio`.

c. Comprensión del protocolo.

- i. Implementar manualmente la inicialización, el descubrimiento y `tools/call` permitió comprender el intercambio real sin depender de FastMCP o de un SDK de MCP.

d. Aplicación del caso industrial.

- i. El servidor local reúne consultas de maquinaria, inventario y órdenes de trabajo. La validación y la confirmación previa reducen el riesgo de acciones accidentales sobre los datos.

2. Comentario personal.

a. Aprendizaje principal.

- i. Esta primera fase ayudó a entender que el LLM no ejecuta directamente las acciones: decide qué herramienta solicitar y utiliza su resultado para preparar la respuesta al usuario.

b. Dificultad encontrada.

- i. La parte más compleja fue coordinar los mensajes por su identificador, mantener el historial de conversación en el formato de la API y conservar `stdout` libre de texto que no pertenezca al protocolo.

c. Continuidad del proyecto.

- i. En la segunda entrega se podrá conservar la lógica del servidor, adaptarla a un transporte remoto y analizar la comunicación con Wireshark. Esas funcionalidades no se presentan como terminadas en esta primera entrega.

3. Alcance de la primera entrega.

a. Contenido incluido.

- i. El proyecto implementa el chatbot, clientes MCP locales, los servidores oficiales configurados y el servidor propio de mantenimiento. El reporte documenta únicamente los incisos 8 y 10, según la aclaración del catedrático.

b. Contenido posterior.

- i. El servidor remoto, el transporte Streamable HTTP, el inciso 9 y el análisis con Wireshark corresponden a la segunda entrega.